Phase 10 — Load Balancing (Complete Notes)

> *The head waiter: distributing traffic, detecting dead servers, enabling zero-downtime everything.*

---

1. What a load balancer does

Sits between clients and N app servers:

1. **Distributes** requests across healthy instances.
2. **Health-checks** and ejects dead/slow instances automatically.
3. Enables **zero-downtime deploys** (drain one node, update, re-add — rolling).
4. Often also: TLS termination, connection reuse to backends, basic DDoS shielding.

2. Layer 4 vs Layer 7 (the core distinction — OSI from Phase 1 pays off)

	L4 (transport)	**L7 (application)**
Sees	IPs + TCP/UDP ports + packets	full HTTP: path, headers, cookies, method
Decides by	connection tuple	**content**: /api→A, /admin→B, Host, headers
Speed	extreme (near wire-speed, no parsing)	slightly slower (parses HTTP, terminates TLS)
Can do	pass-through, preserve TCP	rewrite, redirect, route, retry, WAF, sticky cookies
Examples	AWS **NLB**, NGINX `stream{}`, HAProxy tcp	AWS **ALB**, NGINX `http{}`, HAProxy http

Analogy: L4 = mailroom sorting sealed envelopes by building number only — blazing fast, no reading. L7 = a receptionist who opens each letter and routes by what it says — smarter, can answer some letters herself, slightly slower.

Rule of thumb: **HTTP APIs/web → L7** (you want path routing + smarts). **Databases, MQTT, game servers, raw TCP, millions of conns, need for real client IP at TCP level → L4.**

3. NGINX as a load balancer

Your Phase-5 reverse proxy grows an `upstream` block:

```
upstream api_backend {
    least_conn;                         # algorithm (default: round-robin)
    server 10.0.11.5:8080 max_fails=3 fail_timeout=30s;
    server 10.0.11.6:8080 max_fails=3 fail_timeout=30s;
    server 10.0.11.7:8080 backup;       # only used when others are down
    keepalive 32;                       # reuse TCP connections to backends
}
server {
    listen 443 ssl;
    location /api/ {
        proxy_pass http://api_backend;
        proxy_next_upstream error timeout http_502;   # retry next node on failure
        # + the proxy_set_header lines from Phase 5 (Host, X-Forwarded-For, X-Forwarded-Proto)
    }
}
```

Algorithms

- **round_robin** (default) — 1,2,3,1,2,3... fine when requests are uniform.
- **least_conn** — send to the least busy; better with mixed request durations.
- **ip_hash** — same client IP → same server (sticky by IP; crude session affinity — you know from Phase 9 to prefer Redis sessions instead).
- weights: `server x:8080 weight=3;` — bigger machines take more (also enables canary: weight 95/5).

Health checks: open-source NGINX is **passive** only (`max_fails` — marks a node bad *after* real requests fail). Active checks (probe /health every 5 s) = NGINX Plus, HAProxy, or cloud LBs. Zero-downtime deploy with NGINX LB: remove node from upstream → `nginx -s reload` → deploy node → re-add → reload (or drive it with a script/CI).

4. AWS ALB — Application Load Balancer (L7)

The managed L7 workhorse — what you'll most likely use at work:

```
ALB (multi-AZ by design, scales itself)
├── Listener :443 (TLS terminates here — free ACM cert, auto-renewed)
│   ├── Rule: Host=api.shop.com, Path=/v1/* → Target Group "api-tg"
│   ├── Rule: Path=/admin/*                → Target Group "admin-tg"
│   └── Default                            → api-tg
├── Target Group api-tg:  EC2 instances / ECS tasks / IPs / Lambdas
│   └── Health check: GET /actuator/health every 10s, 2 ok=healthy, 3 fail=out
└── Listener :80 → redirect 301 to :443
```

- **Target group** = the pool + its health check policy. Instances register/deregister dynamically — this is how auto scaling plugs in (ASG adds instance → joins TG when healthy).
- **Connection draining** (deregistration delay): on removal, in-flight requests get e.g. 30 s to finish — zero-dropped-request deploys.
- Sticky sessions available (cookie-based) — again, Redis sessions are the better answer.
- Sends `X-Forwarded-For/Proto` automatically (Phase 5 knowledge applies).
- CloudWatch metrics built in: RequestCount, TargetResponseTime, HTTPCode_Target_5XX, UnHealthyHostCount → alarm on these (Phase 8).
- Pricing: hourly + per-LCU (traffic/connections). Cheap vs running your own pair of NGINX boxes with failover.

5. AWS NLB — Network Load Balancer (L4)

- Routes at TCP/UDP level; **millions of req/s, ~µs latency**; a **static IP per AZ** (Elastic IP attachable — ALB can't do this; firewalls/whitelists love static IPs).

- **Preserves source IP** to the backend natively.
- No path routing, no header logic — it never reads your HTTP.
- Use when: non-HTTP protocols (PostgreSQL proxying, MQTT, game/UDP), extreme throughput, PrivateLink, or TLS pass-through requirements (end-to-end encryption to the pod/instance).
- Common combo: `NLB (static IPs, edge) → ALB or NGINX (L7 logic)`.

6. API Gateway — the L7 sibling with a different job

An **API gateway** (AWS API Gateway, Kong, Spring Cloud Gateway) is "L7 LB + API management brain":

- Per-client **authentication** (API keys, JWT/Cognito validation *before* your app), **rate limiting & quotas** per consumer, request/response transformation, versioning (/v1, /v2), caching, usage plans & billing, single front door for many microservices.
- AWS API Gateway: serverless, pay-per-request — natural fit with Lambda; also proxies to ALB/HTTP backends.

LB vs API gateway in one line: *a load balancer distributes traffic; an API gateway governs it.* Small setups: ALB is enough (NGINX rate limiting covers basics — Phase 5). Many microservices/public APIs/third-party consumers: gateway earns its complexity. They stack: `API Gateway → ALB → instances`.

7. Production examples

- Standard AWS web stack: `Route53 → CloudFront → ALB (ACM TLS, 2+ AZs) → target group → EC2/ECS → RDS`.
- Canary release: ALB weighted target groups (95% v1 / 5% v2) or NGINX weights — watch 5xx/latency metrics, then shift.
- Blue-green: two identical environments; flip the LB (or Route 53 alias) from blue to green; instant rollback = flip back.
- Kubernetes (Phase 11): `NLB/ALB → Ingress (NGINX inside the cluster) → Services → Pods` — the same two-layer L4/L7 pattern, containerized.

8. Common mistakes

- Health check = "port answers" instead of a real /health that verifies DB connectivity → LB routes to zombies. But also: health checks that check *too much* (external APIs) → one vendor blip marks ALL nodes unhealthy → total outage. Check yourself, not the world.
- Forgetting draining → every deploy drops in-flight requests.
- Sticky sessions as the availability strategy (node dies = those users' sessions die — Phase 9).
- LB itself as SPOF (one NGINX VM, no failover) while proudly running 5 app servers.
- Health check interval/threshold too slow (2-min detection = 2-min partial outage) or too twitchy (flapping).
- Timeout mismatch: ALB idle timeout 60 s > app's 30 s → mysterious 502s. Align: LB ≥ app.
- Not passing/reading X-Forwarded-For → rate limiting and logs see one IP: the LB's.

9. Interview questions

1. L4 vs L7 load balancing — what does each see, when do you choose each? (ALB vs NLB concretely.)
2. Round-robin vs least_conn vs ip_hash — trade-offs; why is ip_hash a poor session strategy?
3. How do health checks work? Design a good /health endpoint — what should it check and NOT check?
4. How does a zero-downtime rolling deploy work with a load balancer (draining!)?
5. Explain blue-green vs canary, and how an LB enables each.
6. Load balancer vs API gateway?
7. Client's real IP behind an LB — how do you get it and why does it matter?

8. Users report intermittent 502s after you added the LB — list plausible causes (timeouts, draining, keepalive, health flapping).

10. LAB — build it with NGINX + Docker (extends Phase 9 lab)

```
# 3 instances behind NGINX with least_conn + an /instance endpoint returning HOSTNAME
docker compose up -d --scale api=3

# 1. watch distribution
for i in $(seq 1 12); do curl -s localhost/api/instance; echo; done   # rotation!

# 2. kill a node — LB survives
docker compose stop <one-api-container>
for i in $(seq 1 12); do curl -s localhost/api/instance; echo; done   # only 2 hostnames,
tail -f error.log                                                     # passive health check noted it
# some requests may have eaten one failure first → add proxy_next_upstream, retry: user never sees it

# 3. zero-downtime "deploy"
# remove one server from upstream → docker exec nginx nginx -s reload → restart that api → re-add → reload
# run in parallel:  ab -n 5000 -c 20 localhost/api/instance   → 0 failed requests = success

# 4. weighted canary: weight=9 / weight=1 → count hostnames over 100 curls
```

AWS variant (do after Phase 8 lab): 2× t3.micro in 2 AZs + ALB + target group with /actuator/health → terminate one instance mid-`ab` and watch UnHealthyHostCount + zero downtime.

11. ASSIGNMENT 10 (submit to Claude)

1. Lab evidence: upstream config, the 12-curl outputs (before/after killing a node), and the `ab` summary from the zero-downtime deploy (failed requests must be 0).
2. Design the complete LB layer for: public REST API (10k req/min), an admin panel, a WebSocket service, and a partner-facing API with per-partner quotas. Choose ALB/NLB/NGINX/API-Gateway per component; justify each and draw the diagram.
3. Write the ideal `/actuator/health` behavior for the LB: list what it checks, what it deliberately ignores, and explain the "checking too much" outage mode.
4. Timeout chain: browser(30s) → CloudFront(60s) → ALB(60s) → NGINX(60s) → Spring Boot(?)... A report generation endpoint takes 90 s. What happens today, which component answers what, and how do you fix it properly (two different approaches)?
5. From memory: explain blue-green deployment to a junior with an ALB, including the rollback story and the database-migration caveat.